
3. Type 1 回傳 JSON

```
{  
  "type":1,  
  "elapsed":...,  
  "bdis":...,  
  "fdis":...,  
  "tmp":...,  
  "score":...,  
  "state":...  
}
```

欄位意義

- type
 - 測驗類型
 - 固定為 1
- elapsed
 - 從 start_exercise(1) 開始後，已經經過的秒數
- bdis
 - pts[3] 和 pts[6] 的 2D 距離，再乘上 1000
 - 可理解為某組身體後側或上半身的距離指標
- fdis
 - pts[20] 和 pts[21] 的 2D 距離，再乘上 1000
 - 可理解為某組前側或腳部的距離指標
- tmp

- 連續異常累積次數
 - 若異常持續太久，就判定失敗
- score
 - Type 1 的分數
 - 對應內部變數 `score1_1`
- state
 - 當前測驗狀態
 - 1 代表還在測驗中
 - 0 代表測驗已結束

分數判定標準

在 10 秒 內檢查：

異常條件

只要有任一成立就累積 tmp

- `fdis > 360`
- `bdis > 300`

失敗判定

- 若 `tmp > 25`
 - `score = 0`
 - `state = 0`

成功判定

- 若撐過 10 秒 都沒失敗
 - `score = 1`
 - `state = 0`

Type 1 分數總結

- 0：中途失敗
 - 1：成功維持 10 秒
-

4. Type 2 回傳 JSON

```
{  
  "type":2,  
  "elapsed":...,  
  "bdis":...,  
  "fdis":...,  
  "tmp":...,  
  "score":...,  
  "state":...  
}
```

欄位意義

- type
 - 固定為 2
- elapsed
 - 已經經過的秒數
- bdis
 - pts[3] 和 pts[6] 的 2D 距離 × 1000
- fdis
 - pts[20] 和 pts[21] 的 2D 距離 × 1000
- tmp
 - 連續異常次數

- score
 - Type 2 的分數
 - 對應 score1_2
- state
 - 2 代表進行中
 - 0 代表結束

分數判定標準

在 10 秒 內檢查：

異常條件

- fdis > 380
- 或 bdis > 330

符合則：

- tmp += 1

失敗判定

- 若 tmp > 25
 - score = 0
 - state = 0

成功判定

- 若超過 10 秒
 - score = 1
 - state = 0

Type 2 分數總結

- 0：中途失敗
- 1：成功維持 10 秒

5. Type 3 回傳 JSON

```
{  
  "type":3,  
  "elapsed":...,  
  "bdis":...,  
  "fdis":...,  
  "dd":...,  
  "tmp":...,  
  "score":...,  
  "state":...  
}
```

欄位意義

- type
 - 固定為 3
- elapsed
 - 已經經過的秒數
- bdis
 - pts[3] 和 pts[6] 的 2D 距離 × 1000
- fdis
 - pts[20] 和 pts[21] 的 2D 距離 × 1000
- dd
 - 深度差異值
 - 計算方式：

- $d1 = \text{abs}(\text{pts}[21][2] - \text{pts}[3][2])$
- $d2 = \text{abs}(\text{pts}[20][2] - \text{pts}[6][2])$
- $dd = \text{abs}(d1 - d2)$
- tmp
 - 連續異常次數
- score
 - Type 3 的分數
 - 對應 score1_3
- state
 - 3 表示進行中
 - 之後可能切到 10 或 0

分數判定標準

在 10 秒 內，只要以下任一成立，就算異常：

- $\text{fdis} > 360$
- $\text{bdis} > 300$
- $dd < 0.05$

符合則：

- $\text{tmp} += 1$

當 $\text{tmp} > 25$ 時

立刻結束 Type 3，並切換到 type 10

分數判定

- 若 $\text{elapsed} < 3.0$
 - $\text{score} = 0$
- 若 $\text{elapsed} \geq 3.0$

- score = 1

之後：

- state 會變成 10

若超過 10 秒都還沒觸發

- score = 2
- state = 10

Type 3 分數總結

- 0：太早觸發（小於 3 秒）
 - 1：3 秒後到 10 秒內觸發
 - 2：超過 10 秒才結束
-

6. Type 10 回傳 JSON

```
{  
  
  "type":10,  
  
  "how_far":...,  
  
  "state":0  
}
```

欄位意義

- type
 - 固定為 10
- how_far
 - 取 $\text{pts}[0][2] * 1000$
 - 可理解為目前身體或基準點的 z 軸距離基準值
- state

- 固定為 0
- 因為 type 10 執行完就立即結束

分數判定標準

- Type 10 本身不直接計分
 - 它只是設定後續 Type 4 需要使用的基準距離 how_far
-

7. Type 4 回傳 JSON

```
{  
  
  "type":4,  
  
  "elapsed":...,  
  
  "how_far":...,  
  
  "zpos":...,  
  
  "diff":...,  
  
  "score":...,  
  
  "state":...  
}
```

欄位意義

- type
 - 固定為 4
- elapsed
 - 開始測驗後經過的秒數
- how_far
 - 前面 Type 10 儲存的基準距離
- zpos

- 目前 `pts[0][2] * 1000`
- `diff`
 - 當前距離差
 - 原始計算是：
 - `diff_raw = zpos - how_far`
 - 但 JSON 送出去的是：
 - `diff = diff_raw / 10`
- `score`
 - Type 4 的分數
 - 對應 `score2`
- `state`
 - 4 表示進行中
 - 0 表示結束

分數判定標準

成功條件

若：

- `zpos - how_far >= 3000`

則依時間給分：

- `elapsed < 3.62 → score = 4`
- `elapsed <= 4.65 → score = 3`
- `elapsed <= 6.52 → score = 2`
- `elapsed <= 15.0 → score = 1`
- `elapsed > 15.0 → score = 0`

之後：

- state = 0

超時失敗

若：

- elapsed > 15.0
- 且還沒成功

則：

- score = 0
- state = 0

Type 4 分數總結

- 4：小於 3.62 秒完成
- 3：3.62 ~ 4.65 秒完成
- 2：4.65 ~ 6.52 秒完成
- 1：6.52 ~ 15 秒完成
- 0：15 秒內未完成或超時

8. Type 5 回傳 JSON

```
{  
  "type":5,  
  "elapsed":...,  
  "angle_r":...,  
  "angle_l":...,  
  "maxR":...,  
  "maxL":...,  
  "down":...,
```

```
"sit":...,  
  
"score":...,  
  
"state":...  
}
```

欄位意義

- type
 - 固定為 5
- elapsed
 - 測驗開始後經過的秒數
- angle_r
 - 右側角度
 - `cal_angle(pts[4], pts[5], pts[6])`
- angle_l
 - 左側角度
 - `cal_angle(pts[1], pts[2], pts[3])`
- maxR
 - 右側曾經出現過的最大角度
- maxL
 - 左側曾經出現過的最大角度
- down
 - 是否處於下蹲/下坐階段
 - 1 表示是
 - 0 表示否
- sit

- 已完成的次數
- 對應 sit_count
- score
 - Type 5 的分數
 - 對應 score3
- state
 - 5 代表進行中
 - 0 代表已結束

動作判定標準

進入下蹲狀態

若同時成立：

- $\text{angle_r} < \text{maxR} - 55$
- $\text{angle_l} < \text{maxL} - 55$
- $\text{down} == \text{false}$

則：

- $\text{down} = \text{true}$

完成一次動作

若同時成立：

- $\text{angle_r} > \text{maxR} - 40$
- $\text{angle_l} > \text{maxL} - 40$
- $\text{down} == \text{true}$

則：

- $\text{down} = \text{false}$
- $\text{sit} += 1$

分數判定標準

當：

- $\text{sit} \geq 5$

代表完成測驗，依 `elapsed` 給分：

- $\text{elapsed} < 11.19 \rightarrow \text{score} = 4$
- $\text{elapsed} < 13.69 \rightarrow \text{score} = 3$
- $\text{elapsed} < 16.69 \rightarrow \text{score} = 2$
- $\text{elapsed} < 59.9 \rightarrow \text{score} = 1$
- 否則 $\rightarrow \text{score} = 0$

並且：

- $\text{state} = 0$

超時失敗

若：

- $\text{elapsed} > 60.0$

則：

- $\text{score} = 0$
- $\text{state} = 0$

Type 5 分數總結

- 4：5 次動作在 11.19 秒內完成
 - 3：11.19 ~ 13.69 秒內完成
 - 2：13.69 ~ 16.69 秒內完成
 - 1：16.69 ~ 59.9 秒內完成
 - 0：超過 59.9 秒或超時 60 秒
-

9. get_score() 回傳 JSON

```
{  
  "score1_1":...,  
  "score1_2":...,  
  "score1_3":...,  
  "score2":...,  
  "score3":...,  
  "total":...  
}
```

欄位意義

- score1_1
 - Type 1 的最終分數
- score1_2
 - Type 2 的最終分數
- score1_3
 - Type 3 的最終分數
- score2
 - Type 4 的最終分數
- score3
 - Type 5 的最終分數
- total
 - 總分
 - 計算方式：
 - $\text{score1_1} + \text{score1_2} + \text{score1_3} + \text{score2} + \text{score3}$

10. 各測驗分數範圍總表

Type 1

- 可能分數：0 或 1

Type 2

- 可能分數：0 或 1

Type 3

- 可能分數：0 / 1 / 2

Type 4

- 可能分數：0 / 1 / 2 / 3 / 4

Type 5

- 可能分數：0 / 1 / 2 / 3 / 4
-

11. 最適合另一端直接對接的欄位說明

如果你是要給 Unity / App / 前端那邊看，最重要的是這樣理解：

共通重要欄位

- type：目前是哪一種測驗結果
- elapsed：已花時間
- score：這一關的分數
- state：
 - 非 0 = 還在執行
 - 0 = 這關結束

各類型補充欄位

- Type 1 / 2

- bdis、fdis：距離判斷值
 - tmp：連續異常次數
 - Type 3
 - 多一個 dd：深度差異判斷值
 - Type 4
 - how_far：基準距離
 - zpos：當前距離
 - diff：兩者差值
 - Type 5
 - angle_r、angle_l：左右角度
 - maxR、maxL：歷史最大角度
 - down：是否在下蹲階段
 - sit：已完成次數
-

12. 簡化版分數判定一句話總結

- **Type 1**：撐 10 秒且不要連續超出距離門檻，成功得 1
- **Type 2**：跟 Type 1 類似，但門檻稍微放寬，成功得 1
- **Type 3**：依觸發異常的時間早晚給 0 / 1 / 2
- **Type 4**：達成位移目標後，依完成速度給 0~4
- **Type 5**：完成 5 次動作後，依總時間給 0~4

如果你要，我下一則可以直接幫你整理成 **C#** 可直接建立的 **class** 結構，例如 Type1Result, Type4Result, ScoreResult 這種格式。